

Hands-On Coding Assessment Preparation Guide

Assessment Overview

The Hands-On Coding section is designed to evaluate how well you can understand a problem, recognize the right algorithmic pattern, work within constraints, and implement a clean and efficient solution.

For each problem, pay attention to four things before you start coding:

1. **What is the core problem pattern?**
2. **What do the constraints allow?**
3. **What time and space complexity is expected?**
4. **What edge cases could appear in hidden tests?**

Public vs. Hidden Test Cases

The sample test cases are provided to help you understand the problem and validate your basic implementation.

Passing the public samples does **not** guarantee that the solution will pass the complete evaluation. Your submission will also be evaluated against additional hidden test cases covering boundary conditions, edge cases, larger inputs, and performance.

Design your solution for the **full constraint range**, not just the visible examples.

General Expectations

A strong submission should be:

- Correct
- Efficient
- Readable
- Maintainable
- Robust against edge cases

The expected complexity and time limit are important. A logically correct brute-force solution can still fail if it exceeds the allowed runtime.

Understanding Constraints

Use the constraints to decide how you should approach a problem.

Input Size / Limit	What to Think About
<code>N <= 10^5</code>	Usually target <code>O(N)</code> or <code>O(N log N)</code>
<code>N <= 10^6</code>	Prefer <code>O(N)</code>
Grid up to <code>400 x 400</code>	<code>O(n^2)</code> grid processing is reasonable
<code>0.5 sec</code> time limit	Be especially careful about repeated work and implementation overhead
<code>0.8 sec</code> time limit	Efficient algorithms and efficient I/O are expected

Before coding, ask:

Can this approach handle the maximum input within the given time limit?

Language-Specific Considerations

You may use the language supported by the assessment that you are most comfortable with. However, keep the performance characteristics of your language in mind.

Go

- Use buffered input/output for large inputs.
- Prefer iterative approaches for large sequences and traversals.
- Avoid unnecessary slice allocations or repeated copying.
- Keep helper functions focused and readable.

C / C++

- Use appropriate integer types for the possible value range.
- Avoid unnecessary copying of large arrays or containers.
- Use efficient input/output when required.
- Be careful with indexing and boundary conditions.

Java

- Prefer buffered input for large datasets.
- Be mindful of recursion depth.
- Avoid unnecessary object creation in performance-critical loops.
- Prefer primitive arrays where appropriate.

Python

- Use efficient input methods for large datasets.
- Use appropriate structures such as `deque` for BFS.
- Avoid nested loops that lead to quadratic or worse complexity when the constraints are large.

- Be especially careful with the `0.5` and `0.8` second limits.
-

Code Quality Expectations

Your code should be neat and easy to review.

Aim for

- Meaningful variable names
- Clear state transitions
- Small, focused helper functions where useful
- Consistent formatting
- Minimal duplicated logic
- No unnecessary comments or code
- No debugging prints in the final submission

Avoid

- Unnecessary nested loops
- Repeated calculations
- Large temporary structures when constant-space processing is possible
- Complex logic that can be expressed more clearly
- Special-case hacks that only work for sample inputs

The goal is not to write the shortest code. The goal is to write code that is **clear, correct, and efficient**.

Problem 1: WarriorDivineEnergy

Difficulty: Hard

Expected Time to Solve: 30 minutes

Time Limit: 0.8 seconds

Configured Time Complexity: $O(N)$

Constraints: `1 <= N <= 10^6`, `-10^5 <= arr[i] <= 10^5`

What this question evaluates

This problem evaluates your ability to maximize the total energy of a **continuous sequence** while considering a one-time optional operation that can remove the negative impact of one cursed value.

It mainly tests:

- Dynamic Programming
- Maximum subarray reasoning
- State transitions
- Space optimization

Target Pattern

Think in terms of a **Kadane-style dynamic programming variation**.

The important observation is that there are two situations to keep track of while moving through the array:

- The best result when the special operation has **not** been used.
- The best result when the special operation **has already been used**.

The challenge is maintaining these states correctly without evaluating every possible subarray.

Preparation Hints

- Review the standard maximum-subarray problem first.
- Then think about how one additional optional operation changes the DP state.
- The selected sequence must remain continuous.
- The special operation is optional.
- Consider what happens when every value is negative.
- Think about whether you really need an entire DP array or only the previous state.

Complexity Expectations

The platform specifies:

- **Time Complexity:** $O(N)$
- **Time Limit:** 0.8 seconds

For the intended state-based approach:

- **Time:** $O(N)$ — optimal for the given constraints
- **Space:** $O(1)$ with rolling states

With N as large as 10^6 , avoid any approach that examines all subarrays or repeatedly revisits earlier elements.

What to Watch For in Your Language

Because the input can contain up to one million values:

- Use fast input.
- Avoid unnecessary array copies.
- Avoid storing DP states for every position when only the previous state is needed.
- Make sure numeric types can safely hold accumulated sums.

Hidden-Test Thinking

Before submitting, mentally test:

- One element

- All positive values
 - All negative values
 - A mix of positive and negative values
 - A case where using the special operation helps
 - A case where not using it is better
 - The maximum possible array size
-

Problem 2: Hollow Chain

Difficulty: Medium

Expected Time to Solve: 25 minutes

Time Limit: 0.5 seconds

Configured Time Complexity: $O(N)$

Constraints: $1 \leq n \leq 10^5$, $1 \leq \text{arr}[i] \leq 10^5$

What this question evaluates

This problem evaluates your ability to interpret a mathematical recurrence hidden inside a narrative problem and determine the **minimum valid starting value**.

It mainly tests:

- Mathematical reasoning
- Sequence transformations
- Recurrence thinking
- Optimization

Target Pattern

The strongest way to approach this problem is to look for a **deterministic recurrence** rather than treating every possible starting mana value as a separate simulation.

A useful perspective is to reason **backward**.

Ask:

What value must be available before this chamber so that the rest of the sequence can still be completed safely?

Working backward can reduce what initially looks like a much larger state-space problem to a single running value.

Preparation Hints

- Carefully translate the narrative into a mathematical transformation.
- Pay attention to the difference between being **greater than** the chamber value and being **less than or equal to** it.

- Remember that zero is allowed.
- The value must never become negative.
- Look for a backward recurrence.
- Avoid simulating many possible starting values.

Complexity Expectations

The platform specifies:

- **Time Complexity:** $O(N)$
- **Time Limit:** 0.5 seconds

For the intended recurrence:

- **Time:** $O(N)$ — optimal
- **Space:** $O(1)$ — only a running value is required

Because the time limit is only **0.5 seconds**, avoid unnecessary data structures and repeated simulations.

What to Watch For in Your Language

- Keep the implementation iterative.
- Avoid floating-point calculations where integer reasoning is sufficient.
- Be careful with rounding/ceiling behavior if the mathematical transformation involves division.
- Use appropriate integer types.
- Avoid creating additional arrays unless they are genuinely necessary.

Hidden-Test Thinking

Think about:

- $n = 1$
- All chamber strengths equal
- Increasing strengths
- Decreasing strengths
- The minimum possible valid starting value
- Cases where the running value reaches exactly zero
- Large n

Problem 3: Mission: Escape Through the Infected City

Difficulty: Medium

Expected Time to Solve: 25 minutes

Time Limit: 0.8 seconds

Configured Time Complexity: $O(n^2 \log n)$

Constraints: $1 \leq n \leq 400$, grid values are 0 or 1

What this question evaluates

This problem evaluates your ability to work with grids, Manhattan distances, and a path objective where the quality of a route depends on its **weakest point**.

It mainly tests:

- Grid traversal
- Multi-source BFS
- Path feasibility
- Binary Search on Answer
- Graph reasoning

Target Pattern

Think about the problem in two stages.

Stage 1: Preprocess the Grid

Determine the distance from every cell to the nearest infected cell.

Because there may be many infected cells, consider how all of them can be processed together rather than calculating the nearest infected cell independently for every position.

Stage 2: Check a Safety Threshold

Once each cell has a safety value, ask:

Can I reach the destination while only stepping on cells whose safety value is at least ?

If that question can be answered efficiently, you can search for the largest feasible value of .

Preparation Hints

Review:

- Multi-source BFS
- Standard BFS on a grid
- Binary search on a monotonic answer
- Feasibility checking

Remember that the goal is **not** to find the shortest route.

You are trying to maximize:

the minimum safety value encountered along the route.

Complexity Expectations

The platform specifies:

- **Time Complexity:** $O(n^2 \log n)$
- **Time Limit:** 0.8 seconds

For the intended approach:

- **Time:** $O(n^2 \log n)$
- **Space:** $O(n^2)$

With $n \leq 400$, the grid contains up to 160,000 cells, so an $O(n^2)$ preprocessing step is reasonable. However, repeatedly recomputing distances or performing unnecessarily expensive searches can still exceed the time limit.

What to Watch For in Your Language

- Use an efficient queue for BFS.
- Avoid repeatedly rebuilding large structures unnecessarily.
- Store the precomputed safety information so it can be reused.
- Keep visited-state handling efficient.
- Be conscious of the memory required for multiple $n \times n$ arrays.

Hidden-Test Thinking

Test mentally:

- A 1×1 grid
- Start cell is infected
- Destination is close to an infected cell
- Many infected cells
- Few infected cells
- No obvious straight-line safe route
- Large 400×400 grid

Time, Space & Optimality Summary

Problem	Difficulty	Expected Time	Time Limit	Target Time	Target Space
WarriorDivineEnergy	Hard	30 min	0.8 sec	$O(N)$	$O(1)$
Hollow Chain	Medium	25 min	0.5 sec	$O(N)$	$O(1)$
Mission: Escape Through the Infected City	Medium	25 min	0.8 sec	$O(n^2 \log n)$	$O(n^2)$

How to interpret this

The configured complexity is not just informational. It is a strong signal about what kind of solution is expected.

For example:

- $O(N)$ with $N = 10^6$ → think single-pass / linear processing.
 - $O(N)$ with $N = 10^5$ → avoid repeated simulation.
 - $O(n^2 \log n)$ with $n \leq 400$ → grid-wide preprocessing plus an efficient search strategy is reasonable.
-

How to Think About Hidden Test Cases

Public samples are useful for validating the basic idea, but hidden tests are where robustness matters.

Before submitting, deliberately create your own cases.

Boundary Cases

Use:

- Minimum input size
- Maximum input size
- Minimum values
- Maximum values

Structural Edge Cases

Consider:

- All values having the same sign
- Repeated values
- Single-element inputs
- Empty-looking or completely blocked-looking situations where the problem permits them
- Start/end cells with special values
- Cases where the obvious greedy choice is not optimal

Performance Cases

Ask:

"What would happen if the input were at its maximum allowed size?"

If your code would perform millions or billions of unnecessary operations, reconsider the approach before submitting.

Submission Checklist

Before clicking **Submit**, verify:

- ☐ The code compiles successfully.
 - ☐ Input format matches the problem.
 - ☐ Output format matches the problem.
 - ☐ All required function parameters are handled correctly.
 - ☐ No debugging `print` / `fmt.Println` / logging statements remain.
 - ☐ Time complexity matches the expected target.
 - ☐ Space complexity is reasonable.
 - ☐ Maximum constraints have been considered.
 - ☐ Edge cases have been tested mentally.
 - ☐ Integer overflow has been considered.
 - ☐ Input/output is efficient enough for the time limit.
 - ☐ Variable names are clear.
 - ☐ The code is easy to read and review.
-

Common Reasons Correct Ideas Still Fail

A solution can have the right algorithm and still fail because of implementation details.

Watch for:

Time Limit Exceeded (TLE)

Usually caused by:

- Extra nested loops
- Repeated traversal
- Recomputing information that could be preprocessed
- Inefficient data structures
- Slow input/output

Memory Limit Exceeded (MLE)

Usually caused by:

- Unnecessary copies
- Multiple large arrays
- Storing information that can be computed on demand
- Inefficient object-heavy structures

Wrong Answer

Often caused by:

- Off-by-one errors

- Incorrect initialization
 - Mishandling negative values
 - Incorrect boundary checks
 - Missing the "optional" nature of an operation
 - Confusing minimum-path and maximum-path objectives
-

Recommended Problem-Solving Process

Step 1 — Read the entire question

Do not start coding after reading only the first paragraph.

Step 2 — Read the constraints

Constraints often reveal the expected complexity.

Step 3 — Identify the pattern

Ask whether the problem resembles:

- Dynamic Programming
- Recurrence
- Greedy reasoning
- BFS
- Multi-source BFS
- Binary Search on Answer
- Grid traversal

Step 4 — Estimate complexity

Before implementation, make sure the approach can meet the time limit.

Step 5 — Implement the simplest optimal version

Avoid unnecessary abstractions and data structures.

Step 6 — Test the samples

Use the public test cases to catch basic logic and input/output issues.

Step 7 — Test hidden-case scenarios mentally

Especially maximum sizes and edge cases.

Step 8 — Review before submitting

Check correctness, performance, memory usage, and code cleanliness.

Final Takeaway

The strongest way to approach these problems is to focus on the **underlying pattern**, not the story around it.

WarriorDivineEnergy

Think **maximum subarray + an additional DP state**.

Hollow Chain

Think **mathematical transformation + backward recurrence**.

Mission: Escape Through the Infected City

Think **multi-source grid preprocessing + safety-threshold feasibility + binary search**.

For all three, remember:

Correctness gets you started. Efficient complexity, edge-case handling, and clean implementation get you through the full evaluation.

Read carefully. Think about the constraints. Choose the pattern. Then code.

Good luck and happy coding!